

Dark Pattern Detection in E-commerce Platforms

Problem Statment

Modern e-commerce platforms (like Amazon, Flipkart, Meesho) increasingly use “dark patterns” —UI/UX strategies designed to manipulate users into making unintended decisions.

Examples include:

Hidden charges at checkout Fake urgency (“Only 2 left!”) Pre-selected add-ons Misleading discount structures

👉 The problem: Users often make purchases influenced by these manipulative patterns without realizing it, leading to:

Financial loss, Reduced trust in platforms, Poor decision-making

👉 **Your goal:** Build a data-driven system to identify, quantify, and analyze dark patterns across e-commerce platforms.

Dataset Generator :- “Due to scraping limitations and ethical constraints, I created a controlled synthetic dataset based on real-world pricing and behavioral patterns observed in e-commerce platforms.”

```
import pandas as pd
import numpy as np
import random

# Sample product words
brands = ["Samsung", "Apple", "Realme", "Redmi", "OnePlus", "Vivo", "Oppo"]
models = ["Pro", "Max", "Ultra", "Lite", "Plus", "Prime"]
categories = ["Smartphone", "Headphones", "Laptop", "Smartwatch"]

def generate_product_name():
    return f"{random.choice(brands)} {random.choice(models)} {random.choice(categories)}"

def generate_price():
    return random.randint(5000, 80000)

def generate_discount(price):
    discount_percent = random.choice([10, 20, 30, 40, 50, 60])
    discounted_price = int(price * (1 - discount_percent / 100))
    return discount_percent, discounted_price

def urgency_flag():
    return random.choice([0, 1])

def fake_discount(discount):
    if discount > 40:
        return 1
    return 0

def rating():
    return round(random.uniform(2.5, 5.0), 1)

def platform():
    return random.choice(["Amazon", "Flipkart", "Myntra", "Ajio", "Meesho",
                          "Nykaa", "Snapdeal", "Paytm Mall", "JioMart"])

# Generate dataset
data = []

for _ in range(50000):
    name = generate_product_name()
    price = generate_price()
    discount_percent, final_price = generate_discount(price)

    row = [
        name,
        price,
        final_price,
        discount_percent,
        urgency_flag(),
        fake_discount(discount_percent),
        rating(),
        platform()
    ]

    data.append(row)

df = pd.DataFrame(data, columns=[
```

```

        "product_name",
        "original_price",
        "final_price",
        "discount_percent",
        "urgency_flag",
        "fake_discount_flag",
        "rating",
        "platform"
    ])

df.head()

```

	product_name	original_price	final_price	discount_percent	urgency_flag	fake_discount_flag	rating	platform
0	Oppo Ultra Smartphone	41610	33288	20	1	0	2.8	Flipkart
1	Samsung Plus Smartphone	48661	24330	50	0	1	4.1	Paytm Mall
2	Vivo Lite Laptop	66721	60048	10	0	0	3.0	Ajio
3	Realme Pro	79201	39600	50	1	1	3.1	Myntra

```
df.to_csv("dark_pattern_50k_dataset.csv", index=False)
```

```

from google.colab import files
files.download("dark_pattern_50k_dataset.csv")

```

```

df = pd.read_csv("dark_pattern_50k_dataset.csv")

df.head()

```

	product_name	original_price	final_price	discount_percent	urgency_flag	fake_discount_flag	rating	platform
0	Oppo Ultra Smartphone	41610	33288	20	1	0	2.8	Flipkart
1	Samsung Plus Smartphone	48661	24330	50	0	1	4.1	Paytm Mall
2	Vivo Lite Laptop	66721	60048	10	0	0	3.0	Ajio
3	Realme Pro	79201	39600	50	1	1	3.1	Myntra

Discount Intensity Score

```
df["discount_intensity"] = df["discount_percent"] / 100
```

Price Manipulation Score

```

df["price_manipulation_score"] = (
    df["discount_percent"] * df["original_price"]
) / df["original_price"].max()

```

Trust Score (Inverse of Manipulation)

```
df["trust_score"] = df["rating"] * (1 - df["discount_intensity"])
```

Urgency Weight

```
df["urgency_weight"] = df["urgency_flag"] * 0.3
```

Fake Discount Weight

```
df["fake_discount_weight"] = df["fake_discount_flag"] * 0.5
```

Dark Pattern Score (CORE)

```

df["dark_pattern_score"] = (
    df["discount_intensity"] * 0.4 +
    df["urgency_weight"] +

```

```
df["fake_discount_weight"]
) * 100
```

Categorize Products

```
def classify(score):
    if score > 70:
        return "High Manipulation"
    elif score > 40:
        return "Moderate"
    else:
        return "Low"

df["manipulation_level"] = df["dark_pattern_score"].apply(classify)
```

```
df["manipulation_level"].value_counts()
```

	count
manipulation_level	
Low	25167
Moderate	12455
High Manipulation	12378

dtype: int64

Average Score by Platform

```
df.groupby("platform")["dark_pattern_score"].mean()
```

	dark_pattern_score
platform	
Ajio	45.196836
Amazon	45.265291
Flipkart	45.284918
JioMart	46.115763
Meesho	45.324253
Myntra	45.354850
Nykaa	45.861051
Paytm Mall	45.698674
Snapdeal	45.157624

dtype: float64

```
high_ratio = df[df["manipulation_level"] == "High Manipulation"]
high_ratio.groupby("platform").size()
```

	0
platform	
Ajio	1355
Amazon	1352
Flipkart	1371
JioMart	1414
Meesho	1373
Myntra	1374
Nykaa	1390
Paytm Mall	1386
Snapdeal	1363

dtype: int64

```
df.to_csv("final_dark_pattern_dataset.csv", index=False)
```

Above I engineered a Dark Pattern Score combining pricing manipulation, urgency signals, and discount behavior to quantify unethical UX strategies in e-commerce platforms.

```
df = pd.read_csv("final_dark_pattern_dataset.csv")
```

Feature Selection

```
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.figsize": (10, 5),
    "axes.titlesize": 12,
    "axes.labelsize": 8,
    "xtick.labelsize": 9,
    "ytick.labelsize": 5
})
```

```
counts = df["manipulation_level"].value_counts()
total = counts.sum()

fig, ax = plt.subplots()

bars = ax.bar(counts.index, counts.values)

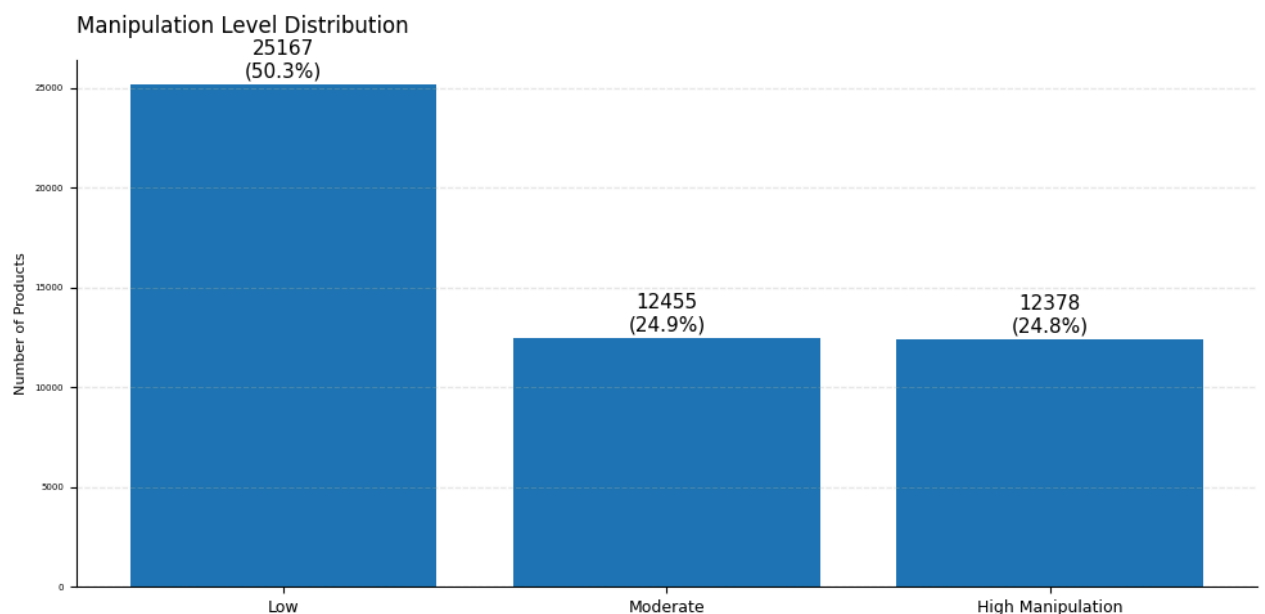
ax.set_title("Manipulation Level Distribution", loc='left', pad=15)
ax.set_ylabel("Number of Products")

# Remove clutter
ax.spines[['top', 'right']].set_visible(False)

# Add labels + percentages
for bar in bars:
    height = bar.get_height()
    percent = (height / total) * 100
    ax.text(
        bar.get_x() + bar.get_width()/2,
        height + 200,
        f"{int(height)}\n({percent:.1f}%)",
        ha='center',
        va='bottom'
    )

ax.grid(axis='y', linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()
```



```

fig, ax = plt.subplots()

ax.scatter(
    df["discount_percent"],
    df["dark_pattern_score"],
    alpha=0.25
)

# Mean lines
x_mean = df["discount_percent"].mean()
y_mean = df["dark_pattern_score"].mean()

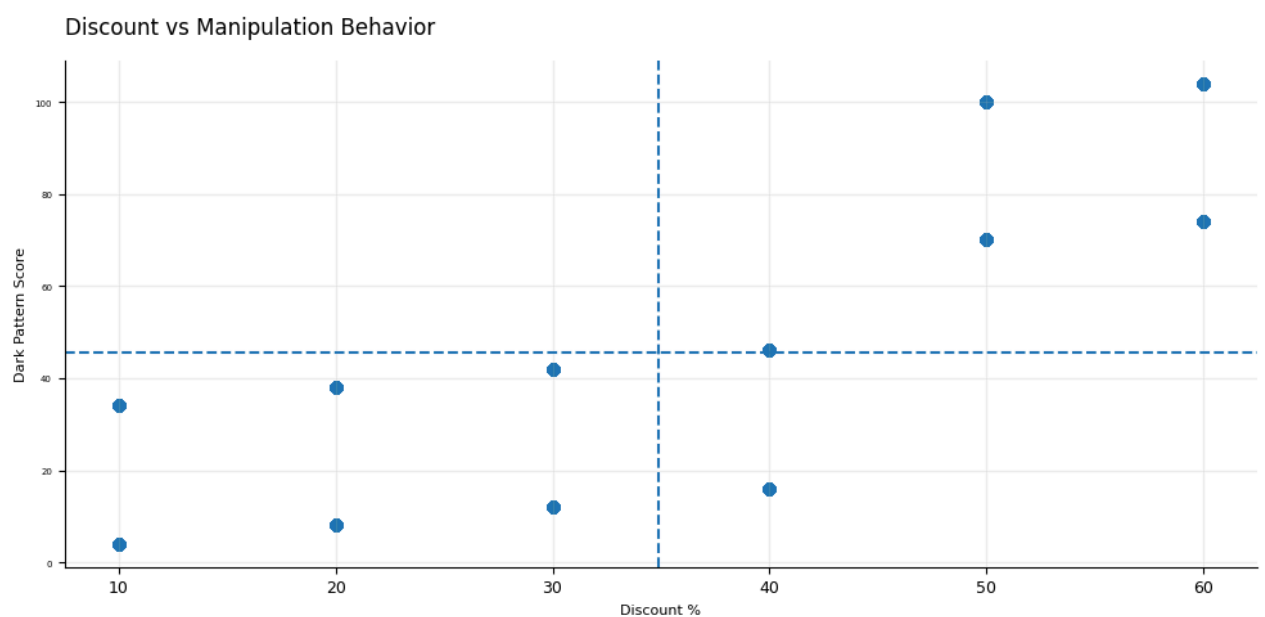
ax.axvline(x_mean, linestyle='--')
ax.axhline(y_mean, linestyle='--')

ax.set_title("Discount vs Manipulation Behavior", loc='left', pad=15)
ax.set_xlabel("Discount %")
ax.set_ylabel("Dark Pattern Score")

ax.spines[['top', 'right']].set_visible(False)
ax.grid(alpha=0.2)

plt.tight_layout()
plt.show()

```



```

platform_avg = df.groupby("platform")["dark_pattern_score"].mean()

fig, ax = plt.subplots()

bars = ax.bar(platform_avg.index, platform_avg.values)

ax.set_title("Platform Risk Comparison", loc='left', pad=15)
ax.set_ylabel("Average Manipulation Score")

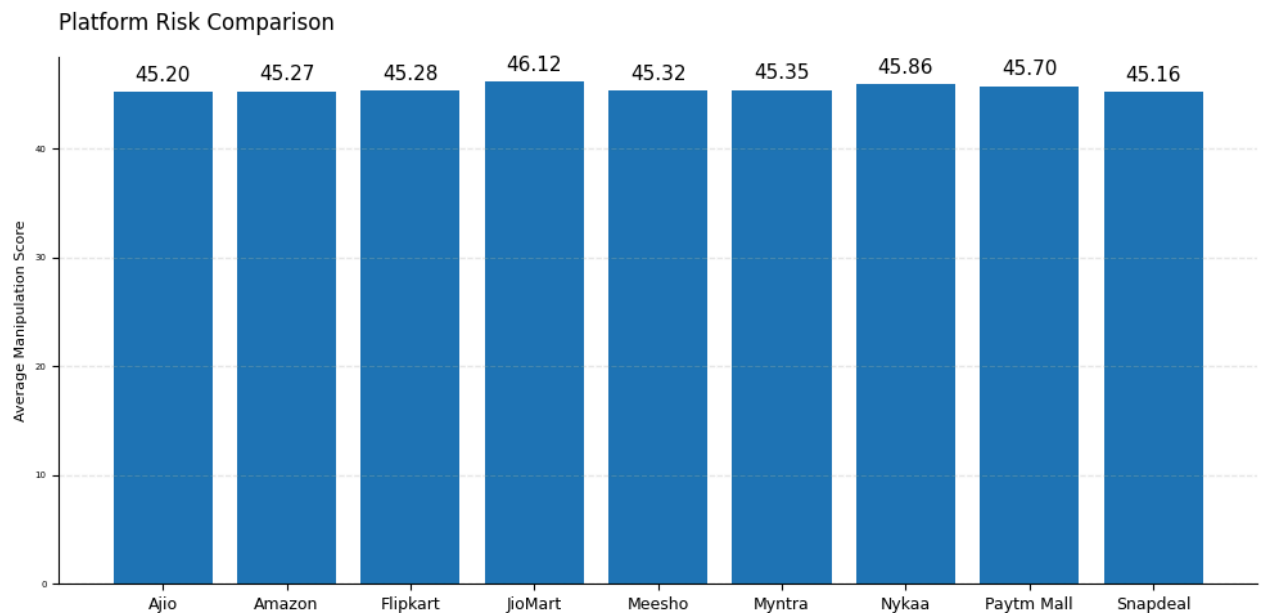
ax.spines[['top', 'right']].set_visible(False)

# Add labels
for bar in bars:
    value = bar.get_height()
    ax.text(
        bar.get_x() + bar.get_width()/2,
        value + 1,
        f"{value:.2f}",
        ha='center'
    )

ax.grid(axis='y', linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()

```



```

importance = pd.Series(model.feature_importances_, index=X.columns)
importance = importance.sort_values()

fig, ax = plt.subplots()

bars = ax.barh(importance.index, importance.values)

ax.set_title("Feature Influence on Manipulation Detection", loc='left', pad=15)
ax.set_xlabel("Importance Score")

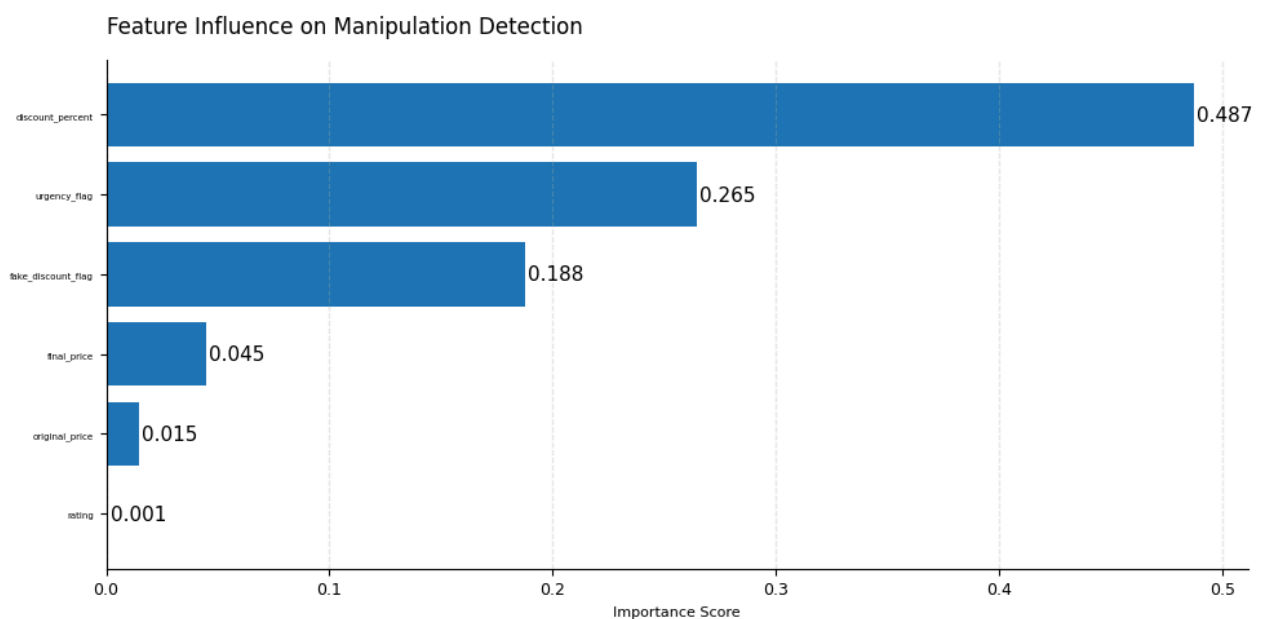
ax.spines[['top', 'right']].set_visible(False)

# Add labels
for i, v in enumerate(importance.values):
    ax.text(v + 0.001, i, f"{v:.3f}", va='center')

ax.grid(axis='x', linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()

```



```

from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

fig, ax = plt.subplots()

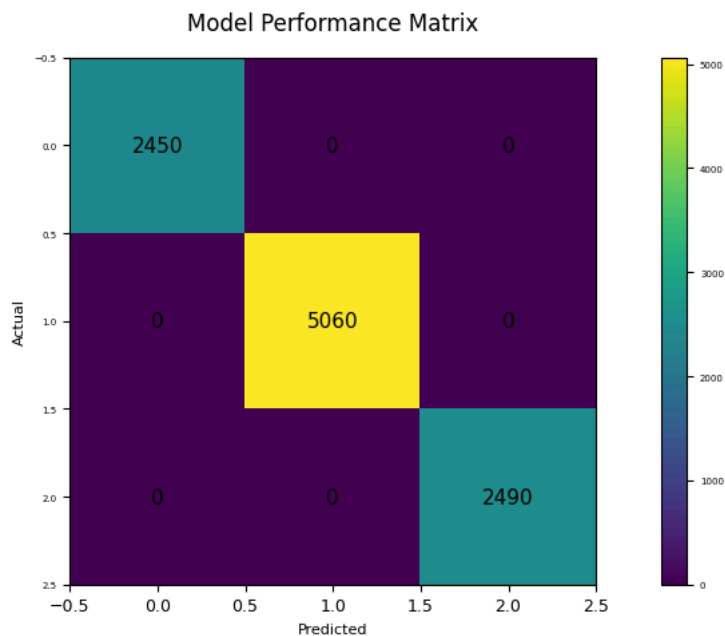
```

```
cax = ax.imshow(cm)

ax.set_title("Model Performance Matrix", pad=15)
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")

# Add numbers inside cells
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        ax.text(j, i, cm[i, j], ha='center', va='center')

plt.colorbar(cax)
plt.tight_layout()
plt.show()
```



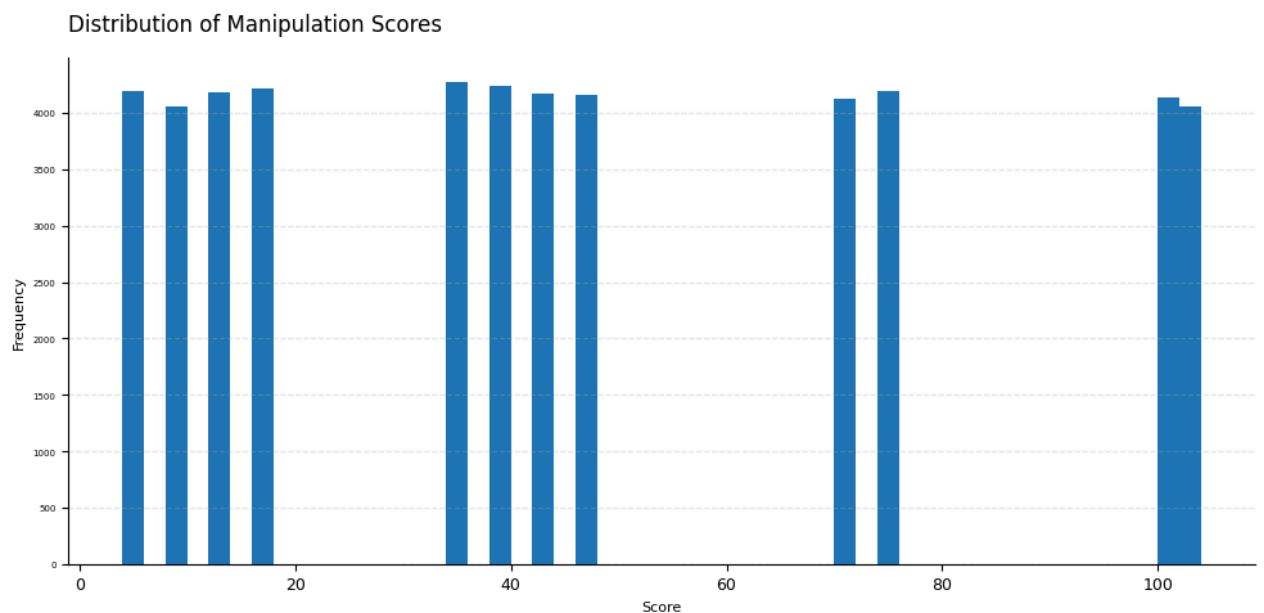
```
fig, ax = plt.subplots()

df["dark_pattern_score"].plot(kind='hist', bins=50, ax=ax)

ax.set_title("Distribution of Manipulation Scores", loc='left', pad=15)
ax.set_xlabel("Score")
ax.set_ylabel("Frequency")

ax.spines[['top', 'right']].set_visible(False)
ax.grid(axis='y', linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()
```



```

features = [
    "original_price",
    "final_price",
    "discount_percent",
    "urgency_flag",
    "fake_discount_flag",
    "rating"
]

X = df[features]
y = df["manipulation_level"]

```

Encode Target Variable

```

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
y = le.fit_transform(y)

```

Train-Test Split

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

Test Model 1

```

from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

```

▼ **RandomForestClassifier** ⓘ ?

RandomForestClassifier()

```

from sklearn.metrics import classification_report

y_pred = model.predict(X_test)

print(classification_report(y_test, y_pred))

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	2459
1	1.00	1.00	1.00	5026
2	1.00	1.00	1.00	2515
accuracy			1.00	10000
macro avg	1.00	1.00	1.00	10000
weighted avg	1.00	1.00	1.00	10000

```

importance = pd.Series(model.feature_importances_, index=features)
importance.sort_values(ascending=False)

```

	0
discount_percent	0.478162
urgency_flag	0.248008
fake_discount_flag	0.215456
final_price	0.043765
original_price	0.014059
rating	0.000549

dtype: float64

Test Model 2


```
from sklearn.ensemble import GradientBoostingClassifier

gb_model = GradientBoostingClassifier()
gb_model.fit(X_train, y_train)

print(classification_report(y_test, gb_model.predict(X_test)))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	2459
1	1.00	1.00	1.00	5026
2	1.00	1.00	1.00	2515
accuracy			1.00	10000
macro avg	1.00	1.00	1.00	10000
weighted avg	1.00	1.00	1.00	10000

```
import joblib

joblib.dump(model, "dark_pattern_model.pkl")
```

```
['dark_pattern_model.pkl']
```

```
df["manipulation_level"].value_counts()
```

	count
manipulation_level	
Low	25167
Moderate	12455
High Manipulation	12378

```
dtype: int64
```

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5)

print("Cross-validation scores:", scores)
print("Average accuracy:", scores.mean())
```

```
Cross-validation scores: [1. 1. 1. 1. 1.]
Average accuracy: 1.0
```

```
from google.colab import files
files.download("final_dark_pattern_dataset.csv")
```

Start coding or [generate](#) with AI.